How Git and GitHub Work Together

1 Comment / By Dot Net Tutorials / February 22, 2026

Back to: Microservices using ASP.NET Core Web API Tutorials

How Git and GitHub Work Together

Before learning GitHub operations, keep one simple idea very clear: Git and GitHub are related, but they are not the same thing. Git is the **Version Control System** that tracks code changes, maintains history, manages branches, and allows us to commit, merge, push, and pull code changes in a structured way. GitHub is an **Online Hosting Platform** that stores Git repositories on the internet so they can be shared, backed up, reviewed, and synchronized across multiple developers.

In day-to-day development:

- We write code on our own computer inside a **local repository**
- We create commits locally using Git
- When we want to share that work, we **push** those commits to GitHub
- When someone else pushes new work to GitHub, we **pull** those changes into our local machine

This is the foundation of real collaboration.

In simple words:

- **Git**: The actual version control engine. It runs on your machine and performs add, commit, branch, merge, pull, and push operations.
- **GitHub**: The online platform that hosts your Git repositories and makes them accessible over the internet.
- **Local repository**: The repository on your computer where you do the actual development work.

Create a New ASP.NET Core Web API Project Locally

First, create the project locally in Visual Studio.

Step-by-Step

1. Open Visual Studio
2. Click **Create a new project**
3. Select **ASP.NET Core Web API**
4. Click **Next**
5. Provide these details:
    - Project name: **ProductCatalog.API**
    - Solution name: **ProductCatalog**
6. Choose your project location

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

7. Keep the normal Web API defaults
8. Click **Create**

At this stage, the project exists only as a normal code project. Git is not yet tracking it. Project creation comes first; Git tracking begins only after we explicitly create a Git repository.

Adding a Project to Git Using Visual Studio

From the top menu,

- Select **Git**
- Click **Create Git Repository**.

Now, Visual Studio opens the **Create a Git repository** dialog. This window allows you to decide **how and where Git should be initialized** for your project.

To create a LOCAL Git repository from this screen, select ONLY this:

1. **Left panel** → click **Local only**
2. **Right panel**:
    - **Local path** → leave as it is
    - **.gitignore template** → Default (VisualStudio)
    - **License template** → None
    - **Add a README.md** → leave unchecked
3. Click **Create**

Add Real Project Files, so the Example Becomes Practical

To make this walkthrough meaningful, let us add a small product API.

Create Product.cs

Create a folder named **Models**, then add **Product.cs**.

```
namespace ProductCatalog.API.Models
{
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; } = string.Empty;
        public decimal Price { get; set; }
    }
}
```

Create IProductService.cs

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Create a folder named **Services**, then add **IProductService.cs**.

```
using ProductCatalog.API.Models;
namespace ProductCatalog.API.Services
{
    public interface IProductService
    {
        List<Product> GetAll();
        Product? GetById(int id);
        Product Add(Product product);
    }
}
```

Create ProductService.cs

Add **ProductService.cs** inside Services.

```
using ProductCatalog.API.Models;
namespace ProductCatalog.API.Services
{
    public class ProductService : IProductService
    {
        private static readonly List<Product> _products = new()
        {
            new Product { Id = 1, Name = "Keyboard", Price = 1500 },
            new Product { Id = 2, Name = "Mouse", Price = 800 }
        };

        public List<Product> GetAll()
        {
            return _products;
        }

        public Product? GetById(int id)
        {
            return _products.FirstOrDefault(p => p.Id == id);
        }

        public Product Add(Product product)
        {
            var nextId = _products.Max(p => p.Id) + 1;
            product.Id = nextId;
            _products.Add(product);
            return product;
        }
    }
}
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Register the Service in the Program.cs

Open Program.cs and register the service.

```
using ProductCatalog.API.Services;
namespace ProductCatalog.API
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            // Add services to the container.
            builder.Services.AddControllers();
            builder.Services.AddEndpointsApiExplorer();
            builder.Services.AddSwaggerGen();

            builder.Services.AddSingleton<IProductService, ProductService>();

            var app = builder.Build();

            // Configure the HTTP request pipeline.
            if (app.Environment.IsDevelopment())
            {
                app.UseSwagger();
                app.UseSwaggerUI();
            }

            app.UseHttpsRedirection();

            app.UseAuthorization();

            app.MapControllers();

            app.Run();
        }
    }
}
```

Create ProductsController.cs

Inside the **Controllers** folder, add **ProductsController.cs**.

```
using Microsoft.AspNetCore.Mvc;
using ProductCatalog.API.Models;
using ProductCatalog.API.Services;
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
namespace ProductCatalog.API.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ProductsController : ControllerBase
    {
        private readonly IProductService _productService;

        public ProductsController(IProductService productService)
        {
            _productService = productService;
        }

        [HttpGet]
        public IActionResult GetAll()
        {
            return Ok(_productService.GetAll());
        }

        [HttpGet("{id:int}")]
        public IActionResult GetById(int id)
        {
            var product = _productService.GetById(id);

            if (product == null)
                return NotFound($"Product with Id {id} not found.");

            return Ok(product);
        }

        [HttpPost]
        public IActionResult Add(Product product)
        {
            var created = _productService.Add(product);
            return CreatedAtAction(nameof(GetById), new { id = created.Id },
created);
        }
    }
}
```

Now build and run the project once to ensure it works.

Make the First Clean Commit

Now the project has meaningful changes. It is time to make the first commit.

Step-by-Step in Visual Studio

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

1. Open **Git Changes**
2. Review the files carefully
3. Stage the correct files
4. Write this commit message: Initial commit – ProductCatalog API setup
5. Click **Commit Staged**

This gives the repository a clean starting point. Your project now has its first stable checkpoint in Git history. The first commit should represent a clean, meaningful starting point for the project.

Add the Existing Project to GitHub

Now we will place the local project on GitHub.

Step 1: Create an Empty Repository on GitHub

Open GitHub in the browser and create a new repository with these settings:

- **Repository name**: ProductCatalog.API
- **Visibility**: Public or Private
- **Add README**: Leave unchecked
- **Add .gitignore**: Leave unchecked
- **Add License**: Optional, usually left blank at this stage

Because the project already exists locally, GitHub should only provide an empty remote repository.

Step 2: Connect Local Repo to GitHub

Copy the HTTPS repository URL (similar to https://github.com/Pranaya-DotNet/ProductCatalog.API.git) from GitHub.

In Visual Studio:

1. Go to **Git → Push to Git Service**
2. Choose **Existing Remote**
3. Paste the GitHub repository URL
4. Confirm the branch is main
5. Click **Push**

Step 3: Verify on GitHub

Open the repository page on GitHub and verify:

- Files are visible under the **Code** tab
- The selected branch is the main

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- Your latest commit appears in the commit history

Adding an existing project to GitHub means creating the project locally first, then connecting that local Git repository to an empty GitHub repository and pushing it.

Pull Requests (PRs) in GitHub

A **Pull Request** is a formal request to merge changes from one branch into another branch, most commonly from a feature branch or bug-fix branch into main. It is not just a merge button. It is a reviewable workflow that allows the team to inspect changes before they become part of the stable codebase. Instead of silently merging code into main, the Pull Request makes the whole change visible to other developers. They can read the changed files, discuss the logic, request improvements, approve them, and then merge them in a controlled way.

Teams use Pull Requests because they add a quality checkpoint before code reaches main. Without that checkpoint, a developer could push incomplete, risky, or incorrect code directly to the stable branch. That can affect the whole team. The application might stop building, APIs may behave incorrectly, or hidden bugs may enter production. PRs reduce that risk by giving the team a chance to review the code before the merge.

Real-time Example:

We will do **two real PR labs**:

- **Lab 1**: Feature PR using feature/add-category-api
- **Lab 2**: Bug-fix PR using bugfix/fix-product-validation

With the above 2 Labs, you will understand:

- How to create and push a branch
- How to open a pr on GitHub
- How to give review comments
- How to use **request changes**
- How to fix feedback and update the same pr
- How to use **approve**
- How to merge
- How to pull merged code back into Visual Studio

Branching: How to Work Safely Without Breaking Main

The main branch should always represent stable, trusted code. New work, such as features, bug fixes, and experiments, should happen in separate branches. Branching allows you to develop independently while preserving the stability of the main branch.

Why Branching Matters

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- You can build a feature without disturbing stable code
- You can fix a bug safely and test it before merging
- Multiple developers can work in parallel
- Half-finished work stays isolated until it is ready

Recommended Naming Convention

- feature/add-teacher-api
- feature/add-authentication
- bugfix/fix-student-validation
- bugfix/fix-login-null-reference
- hotfix/fix-production-payment-error

Where Should We Create the Branch?

A branch is a **Git feature**, not a GitHub-only feature. That means branches are normally created in your **local repository** using Git, and then pushed to GitHub so GitHub can store and display them. In real development, the normal branch flow is always local first.

So, the workflow is:

- Create a branch locally
- Switch to that branch
- Do your development work
- Commit locally
- Push branch to GitHub
- GitHub then shows the same branch online

Lab 1: Create a Feature Branch, Open a Pull Request, Request Changes, Fix It, Approve It, Merge It

Part 1: Open the Existing Project in Visual Studio

1. Open **Visual Studio**
2. Open your existing solution: **ProductCatalog.API**
3. Look at the **bottom-right corner**
4. Confirm the current branch name
5. It should ideally show: main

If it is not the main, click the branch name and switch to main.

Part 2: Pull the Latest main Branch

In Visual Studio:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

1. Go to the top menu
2. Click **Git**
3. Click **Pull**

This updates your local main from GitHub. Do this **before** creating a new PR branch.

Part 3: Create the Feature Branch in Visual Studio

Now, let us add a new API using a feature branch. We want to add a new **Categories API** without directly touching the stable main branch. We will create a feature branch from main so new work stays isolated and does not disturb stable code.

Step-by-Step in Visual Studio

1. Make sure you are on the main
2. Pull the latest changes for main
3. Open **Git Repository** or **Manage Branches** or go to **View → Git Repository**
4. Right-click the main and choose **New Local Branch From…** or **New Branch**
5. Enter a meaningful branch name: **feature/add-category-api**
6. Check **Checkout branch**
7. Click **Create**

Now confirm at the bottom-right that Visual Studio shows: **feature/add-category-api**. That means all your changes will now go into this branch, not into main.

Part 4: Add the Feature Code in Visual Studio

Now we will add a small Categories API.

Step 1: Add Category Model

1. Right-click **Models**
2. Click **Add → Class**
3. Name it: **Category.cs**

Replace the code with this:

```
namespace ProductCatalog.API.Models
{
    public class Category
    {
        public int Id { get; set; }
        public string Title { get; set; } = string.Empty;
    }
}
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Step 2: Add CategoriesController

1. Right-click the **Controllers** folder
2. Click **Add** → **Controller**
3. Choose **API Controller – Empty**
4. Name it: **CategoriesController**

Replace the code with this:

```
using Microsoft.AspNetCore.Mvc;
using ProductCatalog.API.Models;

namespace ProductCatalog.API.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class CategoriesController : ControllerBase
    {
        [HttpGet]
        public IActionResult Get()
        {
            return Ok(new[]
            {
                new Category { Id = 1, Title = "Electronics" },
                new Category { Id = 2, Title = "Accessories" }
            });
        }
    }
}
```

Part 5: Build and Test the Feature Before Commit

Now, verify the feature locally.

Step 1: Build

1. Go to **Build**
2. Click **Build Solution**

Make sure the build succeeds.

Step 2: Run

1. Press **F5** or click **Run**
2. Swagger should open
3. Find this endpoint: GET /api/Categories

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

4. Click **Try it out**
5. Click **Execute**

The feature is now ready to commit.

Part 6: Commit the Feature in Visual Studio

Now commit only this feature work.

Steps

1. Open **Git → Git Changes**
2. You should see:
    - Models/Category.cs
    - Controllers/CategoriesController.cs
3. Review them
4. Stage the files
5. In the commit message box, enter: Add CategoriesController GET endpoint
6. Click **Commit Staged**

Now the feature exists as a commit in your local feature branch. Feature branches should contain only the code related to that feature and should be committed and pushed independently of main.

Part 7: Push the Feature Branch to GitHub

A PR (Pull Request) cannot be created until the branch is available on GitHub.

Steps in Visual Studio

1. Stay on branch: feature/add-category-api
2. Go to **Git**
3. Click **Push**

If this is the first push to the branch, Visual Studio will publish it to GitHub. After the push, the branch is now visible on GitHub.

Part 8: Create the Pull Request on GitHub

Now move to GitHub.

Step 1: Open Repository

1. Open GitHub in the browser
2. Open repository: ProductCatalog.API

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Step 2: Open PR (Pull Request) Creation Screen

GitHub usually shows a banner like: **Compare & pull request**. Click that.

Step 3: Select Branches

Set:

- **base** = main
- **compare** = feature/add-category-api

This means:

- Take code from feature/add-category-api
- Merge it into the main

Part 9: Enter the PR Title and Description

Now fill the Pull Request properly.

PR Title

Add CategoriesController GET endpoint

PR Description

Copy-paste this:

```
Summary:
This PR adds a new CategoriesController to ProductCatalog.API.

What was added:
- Category model
- CategoriesController with GET endpoint
- Sample category response data

Why:
This change introduces a simple categories API so that the product catalog can
later be grouped by category.

Testing:
- Project builds successfully
- Swagger endpoint tested
- GET /api/categories returns expected sample data

Reviewer focus:
Please check controller naming, route design, and response clarity.
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Then click **Create pull request.** Now the PR is open.

What Is Code Review?

Code review is the process where other developers examine your code changes before they are merged. This usually happens inside the Pull Request.

During code review, reviewers check:

- Whether the logic is correct
- Whether the code is readable
- Whether the naming is clear
- Whether there are bugs or risks
- Whether coding standards are followed
- Whether the change should be improved before merging

So, Code review means that other developers read and evaluate your changes before they are merged. Suppose you opened a PR for feature/add-category-api. A reviewer may check:

- Is CategoriesController named correctly?
- Is the route correct?
- Is the response structure clear?
- Is the code simple and readable?
- Should a service layer be added later for consistency?

How Reviewers Give Feedback?

Reviewers usually give feedback directly inside the Pull Request on GitHub. They can review the changed files line by line and leave comments on specific lines of code, or they can provide general feedback on the whole PR.

Typical feedback may include:

- Ask for better naming
- Suggest refactoring
- Point out missing validation
- Highlight logic risks
- Recommend simplification
- Praise a clean approach

Part 10: Review the PR Practically on GitHub

Now, let us simulate a real review.

Steps

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

1. Open the Pull Request
2. Click the **Files changed** tab
3. Open **Category.cs**
4. Hover on the line: public string Title { get; set; } = string.Empty;
5. Click the blue + icon beside the line
6. Add this review comment: Please rename Title to Name so it stays consistent with Product.Name in our project and click comment.
7. Add another comment on CategoriesController.cs if you want: The controller looks clean. Later, we can move sample data to a service for better consistency and click comment.

Now Submit Review

Step 1: Click Submit review

Use the green button at the top-right of the PR page.

Step 2: In the review summary box, write this

Good start. Please rename Category.Title to Name for naming consistency, then this PR can be approved.

Step 3: Select Request changes

This tells GitHub that the PR should not be merged yet because an important change is still needed. GitHub's review system supports submitting a review with comments and then choosing **Request changes** as the review decision.

Step 4: Click Submit review

Now the PR will show that changes were requested. The PR is now officially marked as needing changes. This is how **Request Changes** works practically.

Part 11: Fix the Review Feedback in Visual Studio

Now go back to Visual Studio and fix the reviewer's comment.

Step 1: Stay on the Same Branch

Make sure you are still on: feature/add-category-api

Step 2: Update Category.cs

Change this:

```
namespace ProductCatalog.API.Models
{
    public class Category
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    {
        public int Id { get; set; }
        public string Name { get; set; } = string.Empty;
    }
}
```

Step 3: Update CategoriesController.cs

Change this:

```
using Microsoft.AspNetCore.Mvc;
using ProductCatalog.API.Models;

namespace ProductCatalog.API.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class CategoriesController : ControllerBase
    {
        [HttpGet]
        public IActionResult Get()
        {
            return Ok(new[]
            {
                new Category { Id = 1, Name = "Electronics" },
                new Category { Id = 2, Name = "Accessories" }
            });
        }
    }
}
```

Step 4: Build and Test Again

1. Build solution
2. Run Swagger
3. Test: GET /api/Categories

Part 12: Commit and Push the Feedback Fix

Now commit the fix.

Steps in Visual Studio

1. Open **Git Changes**
2. Stage the modified files
3. Use this commit message: Rename Category.Title to Name for consistency
4. Click **Commit Staged**

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

5. Click **Push**

Now go back to the same Pull Request on GitHub.

You will see:

- The PR has updated automatically
- The new commit appears in the PR
- Reviewers can review again

This is very important: **You do not create a new PR**. You keep working in the **same branch**, and the **same PR updates automatically**.

What Does Approve and Request Changes Mean?

When reviewing a Pull Request, reviewers usually have different review options.

1. **Approve:** The reviewer believes the PR is ready to be merged. It means the code looks acceptable, or at least no blocking issues remain.
2. **Request Changes:** It means the reviewer believes the PR should not be merged yet. There are important issues that must be fixed first. This does not mean the PR is bad. It only means the code needs improvements before it is safe or appropriate to merge.

Part 13: Approve the Pull Request on GitHub

Now, let us complete the review properly.

Steps

1. Go back to the same Pull Request in GitHub
2. Click **Files changed**
3. Verify the updated code
4. Click **Review changes**
5. In the review comment box, type: The requested naming change has been applied. Looks good now.
6. Select: Approve
7. Click **Submit review**

Now the PR is approved. This is how **Approve** works practically.

Part 14: Check Before Merging

Before merging a Pull Request, the team should verify that the code is truly ready.

Checklist Before Merging

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- Did the project build successfully?
- Have tests passed?
- Have reviewer comments been addressed?
- Is the branch up to date with main?
- Are merge conflicts resolved?
- Is the code logically correct?
- Are naming and coding standards acceptable?
- Is the PR description clear enough?
- Is any secret or sensitive data accidentally included?
- Is the feature or fix complete enough for main?

For our example, confirm:

- Category.cs is correct
- CategoriesController.cs is correct
- Swagger test worked
- The reviewer's comment was fixed
- PR is approved

Part 15: Merge the Pull Request in GitHub

Now merge the PR.

Steps

1. Open the approved Pull Request
2. Click: Merge pull request
3. Then click: Confirm merge
4. Now the branch changes are merged into: main
5. After the merge, GitHub may show: Delete branch. Click that if the feature branch is no longer needed.

Now the main contains:

- Category.cs
- CategoriesController.cs

This is what happens after PR merge, practically.

Part 16: Update Your Local main in Visual Studio After Merge

Merging on GitHub does **not** automatically update your local Visual Studio project.

You must update the local main.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Steps

1. Go back to Visual Studio
2. Switch branch to: main
3. Go to **Git**
4. Click **Pull**

Now, the local main gets the merged code from GitHub.

Optional: Delete Local Branch

If done:

1. Open **View** → **Git Repository**
2. Right-click: feature/add-category-api
3. Click **Delete**

Now your local project is fully synced with the merged PR.

Lab 2: Create and Merge a Bug-Fix Pull Request

Now, let us do a second real PR, but this time for a bug fix. We want to add validation in the POST action of ProductsController. This makes the workflow even clearer.

Part 17: Create the Bug-Fix Branch in Visual Studio

Steps

1. Switch to main
2. Pull the latest main
3. Open **Git Repository**
4. Right-click main
5. Click **New Branch**
6. Use this name: **bugfix/fix-product-validation**
7. Check **Checkout branch**
8. Click **Create**

Now you are on the bug-fix branch.

Part 18: Add Validation in ProductsController

Open ProductsController.cs and update the POST action. Replace the Add method with this:

```
[HttpPost]
public IActionResult Add(Product product)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
{
    if (string.IsNullOrWhiteSpace(product.Name))
    {
        return BadRequest("Product name is required.");
    }

    if (product.Price <= 0)
    {
        return BadRequest("Product price must be greater than zero.");
    }

    var created = _productService.Add(product);
    return CreatedAtAction(nameof(GetById), new { id = created.Id }, created);
}
```

Test It

Run Swagger and test:

- POST with empty name
- POST with price 0
- POST with valid data

Make sure validation works correctly.

Part 19: Commit and Push the Bug-Fix Branch

1. **Commit Message:** Add validation to product POST action
2. Go to **Git → Push**

Now the branch is on GitHub.

Part 20: Create the Bug-Fix PR on GitHub

PR Title

Add validation to the product POST action

PR Description

```
Summary:
This PR adds validation to the POST action in ProductsController.

What was changed:
- Product name is now required
- Product price must be greater than zero
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
Why:
This prevents invalid product data from being added to the API.

Testing:
- Project builds successfully
- Swagger tested
- Empty name returns BadRequest
- Price <= 0 returns BadRequest
- Valid request works correctly

Reviewer focus:
Please verify validation logic and response messages.
```

Then click **Create pull request.** Now the PR is open.

Part 21: Review, Approve, and Merge the Bug-Fix PR

Now repeat the exact same PR cycle:

1. Reviewer opens PR
2. Checks **Files changed**
3. Adds any comments if required
4. Clicks **Approve**
5. Merge PR
6. Delete the branch if finished

Then in Visual Studio:

1. Switch to main
2. Click **Pull**

Now your local main contains both:

- Categories feature
- Product validation bug fix

What Is a Merge Conflict?

A merge conflict happens when Git cannot automatically decide how to combine changes from two branches. This usually happens when both branches modify the same file and the same lines, or nearby lines, in different ways. Git is not saying your project is broken. It is simply asking for your decision.

Common Conflict Areas in ASP.NET Core Projects

- Program.cs when multiple developers add services or middleware

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- Controllers when the same action method is changed differently
- DTOs or models when multiple branches add or rename properties
- appsettings.json when configuration changes overlap
- Dependency injection registrations when multiple features update services

Simulate a Real Merge Conflict in the Program.cs

Now, let us create a practical conflict.

Change in Main Branch

Suppose, in the main, you modify Program.cs to disable camelCase JSON naming.

```
builder.Services.AddControllers()
    .AddJsonOptions(options =>
    {
        options.JsonSerializerOptions.PropertyNamingPolicy = null;
    });
```

Commit this in the main with the message: Configure JSON serialization to keep C# property names

Change in Bug-Fix Branch

Now switch to bugfix/fix-product-validation and modify Program.cs differently to add a CORS policy.

```
builder.Services.AddControllers();

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowAll", policy =>
    {
        policy.AllowAnyOrigin()
              .AllowAnyMethod()
              .AllowAnyHeader();
    });
});
```

Also add:

app.UseCors(“AllowAll”);

Commit this branch with the message: Add CORS policy to allow all origins. Now, both branches have changed Program.cs in overlapping areas. This sets up a realistic conflict.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Note: A merge conflict occurs when two branches change the same file, and Git cannot automatically determine the final code. A conflict is not a Git failure. It is a decision point. Read the code, understand the business requirement, and create the correct final combined result.

How to Resolve Merge Conflicts Practically

When Visual Studio detects a conflict, it opens a merge editor. Typically, you will see **Current**, **Incoming**, and **Result** areas.

- **Current**: The version in the branch you are currently on
- **Incoming**: The version from the branch being merged
- **Result**: The final code that you must approve

Choose:

- **Current,** when the code is already in your current branch, it is correct
- **Incoming** when the code from the branch being merged is the correct one
- **Both** are only needed when both changes are needed and compatible
- If needed, manually edit the **Result** pane instead of blindly accepting one side

Bring the latest main into a Feature Branch

Long-running branches can drift away from the main. To reduce bigger conflicts later, update your branch regularly by bringing the main into it. This helps you discover issues early while the branch is still small.

Visual Studio Flow

- Switch to your feature branch
- Open **Git Repository**
- Right-click main
- Choose **Merge into Current Branch**

If conflicts arise, resolve them immediately rather than postpone them. Updating a branch regularly with main helps detect conflicts earlier, when they are still small and easier to fix.

How to Clone a Project from GitHub

Cloning is used when the repository already exists on GitHub, and you want a full working copy on your machine. This is common when you move to another laptop or join an existing project.

Clone Using Visual Studio

1. Open Visual Studio
2. Go to **Git** → **Clone Repository**

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

3. Paste the repository URL
4. Choose the destination folder on your machine
5. Click **Clone**
6. Open the solution after the download completes

What Clone Gives You

- All project files
- Complete commit history
- Remote connection information
- Branches references
- A fully working local Git repository

What Happens After Cloning

After the clone completes, you can build the application, create branches, commit locally, pull new changes, and push your own work if you have permission. Clone is not a one-time file download. It creates a complete Git-enabled working repository.

Common Beginner Mistakes and How to Avoid Them

- **Working directly on main:** Create a dedicated feature or bugfix branch for every real change.
- **Pushing without pulling first:** Pull the latest changes at the start of work and, if needed, before the final push.
- **Huge commits:** Commit small logical units of work with clear messages.
- **Committing generated files:** Use .gitignore properly and review files before the first push.
- **Blind conflict resolution:** Read the code carefully and produce the correct final result rather than blindly accepting one side.
- **Forgetting to test after merge:** Build, run, and test the project after every meaningful merge.
- **Committing secrets:** Never commit passwords, access keys, PATs, or production connection strings.

Conclusion:

GitHub becomes much easier to understand when the learner sees it in a real project flow rather than as isolated commands. In this practical example, we created a new ASP.NET Core Web API project, set it up as a local Git repository, made the first commit, connected it to GitHub, pushed it to GitHub, and then continued professional development using a feature branch, a bug-fix branch, merge operations, and conflict resolution. That is the real development lifecycle that beginners need to understand.

From here onward, the routine becomes systematic: pull the latest changes, make small logical updates, stage carefully, commit clearly, push regularly, create branches for meaningful work, merge with care, resolve conflicts intelligently, and test after every important merge. Once that discipline becomes a habit, Git and GitHub stop feeling complicated and become safe, reliable parts of professional software development.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent

Do Not Sell My Information

Explore our developer-friendly HTML to PDF API

Printed using PDFCrowd

HTML to PDF